

Task 2 Part C: Counting Bloom Filter Analysis

Issues with Deletion in Counting Bloom Filters

The main issues with deletion in counting Bloom filters arise from the probabilistic nature of the data structure and the shared counters:

1. **False Negatives:** When deleting an item that shares bucket counters with other items, we might accidentally decrease counters that are also being used by other items, potentially creating false negatives. This violates the Bloom filter's guarantee of never having false negatives.
2. **Counter Limitations:** With c -bit counters, we can only count up to $2^c - 1$ items per bucket. This creates an upper bound on how many items can share the same bucket, potentially leading to overflow situations when the filter becomes too full.
3. **Deletion Ambiguity:** Since multiple items can share the same bucket counters, we cannot be certain that decreasing a counter won't affect other items that legitimately exist in the filter.

Complexity Analysis

Let:

- n = number of items in the filter
- k = number of hash functions
- m = number of buckets
- c = number of bits per counter (variable)

Space Complexity

- Standard Bloom Filter: $O(m)$ bits
- Counting Bloom Filter: $O(m * c)$ bits
 - Each bucket now requires c bits instead of 1 bit
 - Total space increased by factor of c

Time Complexity

- Insertion: $O(k)$ operations
 - Must compute k hashes and update k counters
 - Each counter update now requires $O(c)$ bit operations
 - Total: $O(k * c)$
- Deletion: $O(k * c)$
 - Must compute k hashes and update k counters
 - Each counter update requires $O(c)$ bit operations
 - Additional checks for underflow
- Query: $O(k)$

- Same as standard Bloom filter
- Reading c-bit counters doesn't significantly impact asymptotic complexity

Overall Impact

The modification from Part A to Part B introduces a factor of c to both space complexity and update operations (insert/delete), while maintaining the same query complexity. The practical impact depends on the chosen value of c , with $c = 4$ being common but potentially insufficient for some applications requiring higher counts.

Task 2 Part E: Dynamic Bloom Filter Analysis

Complexity Changes from Basic to Dynamic Bloom Filter

The transition from a basic Bloom filter to a dynamic Bloom filter introduces several significant changes in complexity:

Space Complexity

- Basic Bloom Filter: $O(m)$ bits where m is the number of buckets
- Dynamic Bloom Filter: $O(s * m * c)$ where:
 - s is the number of filters
 - m is the number of buckets per filter
 - c is bits per counter (constant) The space complexity increases linearly with the number of filters, but this allows for unlimited growth.

Time Complexity

- Insertion: $O(k) \rightarrow O(k)$
 - Remains constant as we only insert into the latest filter
 - Filter creation ($O(1)$) amortized over insertions
- Lookup: $O(k) \rightarrow O(k + s)$
 - Must check $O(s)$ filters in worst case
 - Each filter requires $O(k)$ hash computations
 - Using `birdNames` array for $O(s)$ tracking improves average case
- Deletion: $O(k) \rightarrow O(k + s)$
 - $O(s)$ to find the correct filter using `birdNames` array
 - $O(k)$ to perform the actual deletion in that filter

Implementation of $O(k + s)$ Deletion

Our implementation achieves $O(k + s)$ deletion by:

1. Using an $O(s)$ space `birdNames` array to track which filter contains each bird
2. Finding the correct filter in $O(s)$ time by scanning this array
3. Performing the actual deletion in $O(k)$ time once the correct filter is found

We assume `BUCKET_SIZE` and `NUM_HASHES` are constants, similar to assumptions made for hash tables where we treat the load factor as constant.

